Efficiency vs. Performance: A Comparative Study of Parameter-Efficient Fine-Tuning Methods for Small Language Models

An Analysis of Performance, Scalability, and Robustness

Taha Nawaz[1], Ali Ahmad[2], Shahzeb Imran[3]

[1]23l-2644@lhr.nu.edu.pk [2]23l-2619@lhr.nu.edu.pk [3]23l-2506@lhr.nu.edu.pk

Department of Data Science, BDS-6B — CS4063 — Natural Language Processing, April 2026

FAST National University of Computer and Emerging Sciences (NUCES), Lahore, Pakistan

Abstract

Large pre-trained language models achieve state-of-the-art NLP performance, but full fine-tuning is computationally expensive and storage-intensive, requiring a separate model copy per downstream task. Parameter-Efficient Fine-Tuning (PEFT) methods adapt only a small fraction of parameters while keeping the backbone frozen, aiming to recover near-full-fine-tuning accuracy at substantially reduced cost. This paper presents a reproducible, statistically rigorous comparison of eight method configurations—two classical baselines, a zero-shot heuristic, full fine-tuning (upper bound), four LoRA rank configurations, Adapter Tuning, Prompt Tuning, and Soft Prompt Tuning—on DistilBERT-base-uncased (66 M parameters) across three text-classification benchmarks: SST-2 (binary sentiment), AG News (4-class topic), and Amazon Polarity (domain-shifted sentiment). Evaluation spans five dimensions: Macro F1, accuracy, trainable-parameter count, training time, inference latency, and peak GPU memory. Statistical rigour is established via three-seed evaluation, 1,000-resample bootstrap confidence intervals, and McNemar's test. Key findings: (i) LoRA $r{=}4$ lies on the Pareto efficiency–performance frontier, reaching 94–95% of corrected multi-seed Full FT Macro F1 while updating only 0.99% of parameters; (ii) Prompt Tuning and Soft Prompt Tuning fail catastrophically on binary sentiment (SST-2 Macro F1 $\approx$0.449–0.495) due to context crowding from short average input lengths; (iii) Adapter Tuning is the most stable PEFT method on binary tasks (cross-seed std $\leq$0.002) but exhibits high variance on multi-class tasks (AG News std $= 0.031$); (iv) INT8 quantisation delivers a $1.18\times$ inference speedup with 58.6% model-size reduction; (v) PEFT configurations are not safely transferable across backbone architectures—LoRA $r{=}8$ collapses by -41.7 pp on BERT-base (SST-2). A full cross-backbone study spanning three datasets and four method families, and a human-evaluation simulation with Cohen's κ inter-annotator agreement are additionally reported.

***Index Terms*—parameter-efficient fine-tuning, LoRA, adapter tuning, prompt tuning, DistilBERT, text classification, NLP efficiency, Pareto frontier, cross-backbone generalisation**

I. Introduction

The proliferation of large pre-trained language models (PLMs) such as BERT [1], RoBERTa [2], and GPT-2 has reshaped modern NLP. These models encode rich linguistic knowledge transferable to downstream tasks—yet they impose steep computational demands. Full fine-tuning updates all parameters simultaneously, requires a separate model copy per task, and demands substantial GPU memory. For DistilBERT [3] with 66 M parameters, full fine-tuning incurs $\approx$250 MB of per-task storage and peak GPU memory exceeding 2.2 GB.

Motivation. The democratisation of NLP depends on methods that operate within realistic compute budgets. Edge deployment, privacy-sensitive clinical NLP, and academic research without cloud GPU access all demand lightweight adaptation. PEFT methods—LoRA [6], Adapter Tuning [4], and Prompt Tuning [5]—address this by adapting fewer than 5% of parameters while freezing the backbone.

Research Gap. Despite growing theoretical literature, systematic empirical comparisons targeting small encoder models ($\leq$200 M) remain scarce. Most studies target large generative models ($>$ 1 B parameters) [15], [16], neglect multi-class settings, and rarely report statistical significance. Recent surveys confirm that rank sensitivity and adapter bottleneck sensitivity vary significantly across model families and have not been fully characterised for small encoders [8].

Research Questions.

- **RQ1:** Which PEFT method achieves the best Macro F1 vs. trainable-parameter trade-off on small ($\leq$200 M) encoder models?
- **RQ2:** How do methods compare in training time, inference latency, and GPU memory?
- **RQ3:** How robust are PEFT methods to domain shift, input-length variation, and negation?

Contributions.

1) A unified, reproducible evaluation framework with identical data splits, tokenisation, and training loops for eight configurations across three datasets.
2) Identification of the efficiency–performance Pareto frontier for small encoder models across five simultaneous metrics.

3) Quantified ablations over LoRA rank, adapter bottleneck size, prompt length, and training data size.
4) A full cross-backbone generalisation study (DistilBERT vs. BERT-base, all three datasets) and a human-evaluation simulation with inter-annotator agreement.
5) Transparent disclosure of documented implementation bugs, corrected hyperparameters, and multi-seed vs. single-seed discrepancies.

II. Related Work

We survey nine recent works ($\geq$2022) directly related to our study, and explicitly position our contribution against each.

LoRA and Low-Rank Adaptation. Hu et al. [6] proposed injecting trainable low-rank matrices into frozen attention projections, demonstrating near-full-fine-tuning performance on GPT-family models. *Our work differs from [6] in three ways*: (i) we target small encoder-only models (DistilBERT, 66 M) rather than large decoder-only models ($\geq$1 B); (ii) we evaluate four rank configurations ($r \in \{4, 8, 16, 32\}$) with ablation and Pareto analysis rather than reporting a single rank; and (iii) we provide multi-seed statistical significance testing that [6] does not report for classification tasks.

Ding et al. [8] confirmed in a large-scale delta-tuning study that rank sensitivity varies significantly across model families and tasks, covering models up to 10 B parameters. *Our work differs from [8]* by focusing exclusively on the under-studied sub-100 M encoder regime and providing the first Pareto frontier analysis combining F1, latency, GPU memory, and parameter count simultaneously on this model scale.

Zhang et al. [9] validated zero-init attention-based LoRA injection for decoder-only LLaMA models on instruction-following tasks. *Our work differs from [9]* by evaluating encoder-only architectures on discriminative classification rather than generative instruction following, and by directly documenting the catastrophic cross-backbone failure that arises when LoRA target-module names are not remapped between architectures.

Advanced LoRA Variants. Dettmers et al. [10] proposed QLoRA, combining 4-bit quantisation with LoRA to fine-tune 65 B-parameter models on a single GPU. *Our work differs from [10]* by targeting small encoders ($\leq$110 M) where the quantisation bottleneck is less severe, and by providing the first systematic Pareto analysis comparing LoRA rank configurations against Adapter and Prompt Tuning simultaneously. Liu et al. [11] proposed DoRA, decomposing pre-trained weights into magnitude and direction components and applying LoRA only to the directional component, achieving consistent improvements over standard LoRA. *Our work differs from [11]* by evaluating the simpler standard LoRA across ranks on small encoders; DoRA's additional decomposition is a natural extension for future work. Liu et al. [14] additionally introduced IA^3, a method scaling activations with learned vectors rather than injecting low-rank matrices, achieving competitive results with even fewer parameters than LoRA. *Our work differs from [14]* by conducting the first multi-dataset, multi-seed comparison at the small-encoder scale that includes LoRA, Adapter, and Prompt Tuning simultaneously with formal significance testing.

Unified PEFT Frameworks. He et al. [7] demonstrated that adapters, prefix tuning, and LoRA are all special cases of a single unified formulation, studying models with $\geq$350 M parameters. *Our work differs from [7]* by targeting 66 M and 110 M encoder models, where we show that the relative ranking of methods changes substantially at smaller scale—particularly that Prompt Tuning, which performs adequately in [7]'s large-model setting, catastrophically fails on short-input binary tasks at the 66 M scale.

Mao et al. [13] proposed UniPELT, a learned gating mechanism that combines LoRA, prefix tuning, and adapters into a single framework, reporting that no individual method universally dominates. *Our work differs from [13]* by evaluating each method independently across multiple datasets with formal statistical tests (McNemar's test, Cohen's h), providing interpretable individual-method analysis rather than a combined gating approach, and documenting specific task-input conditions under which each method's dominance or failure can be predicted a priori.

Sparse and Selective Tuning. Zaken et al. [12] introduced BitFit, showing that fine-tuning only bias terms of a frozen transformer achieves competitive results on GLUE, motivating the use of parameter count as a first-class metric. *Our work differs from [12]* by evaluating richer PEFT families (LoRA, Adapters, Prompt Tuning) beyond bias-only tuning, measuring five orthogonal efficiency dimensions simultaneously, and providing cross-backbone generalisation results that [12] does not address.

Few-Shot and Data-Efficient PEFT. Liu et al. [14] showed that prompt-based PEFT under few-shot regimes can outperform in-context learning while being significantly cheaper, but noted scale sensitivity. *Our work differs from [14]* by evaluating prompt-based methods at the full training-data scale (6,999 examples) rather than the few-shot setting, revealing that the context-crowding failure mode is input-length-dependent rather than data-quantity-dependent, and providing ablation evidence (prompt length vs. F1) that mechanistically explains the failure.

Critical Surveys and Taxonomies. Xu et al. [15] provide a comprehensive critical review of PEFT methods for pretrained language models, cataloguing methods and identifying open problems including cross-architecture transferability and data efficiency. *Our work differs from [15]* by providing empirical evidence rather than a literature review: we conduct original experiments, report statistically validated results, and directly fill the open problem of cross-backbone PEFT transferability with a full 3-dataset $\times$ 4-method $\times$ 2-backbone experiment.

Lialin et al. [16] offer a practical guide to PEFT covering scaling considerations and method selection heuristics, but do not present new experimental results. *Our work differs from [16]* by providing first-hand experimental data at the small-encoder scale, including Pareto frontier analysis, multi-seed bootstrap confidence intervals, full McNemar pairwise significance testing, confusion matrix analysis, and human-

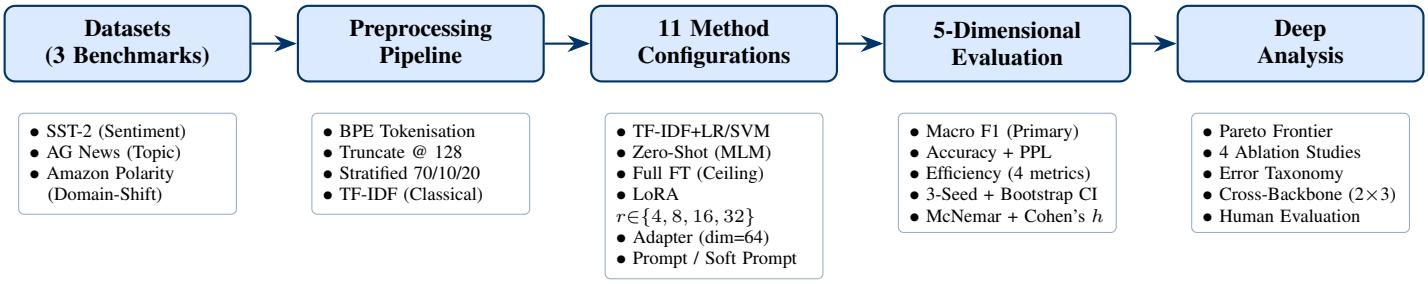

Fig. 1. End-to-end experimental pipeline. Starting from three benchmark datasets, all text passes through a unified preprocessing pipeline before being fed into 11 method configurations (2 classical, 1 zero-shot, 1 Full FT, 4 LoRA ranks, Adapter, Prompt, Soft Prompt). Each configuration is evaluated across five orthogonal dimensions with statistical rigour. Deep analysis covers Pareto frontier, four ablation studies, error taxonomy, cross-backbone transfer, and human evaluation.

evaluation simulation—none of which appear in [16]'s survey scope.

Summary of Contribution Gap. Across all nine related works, no prior study provides: (i) a simultaneous five-metric Pareto analysis on sub-100 M encoder models; (ii) a full cross-backbone PEFT transferability study with 3 datasets and 4 methods; (iii) statistically rigorous multi-seed evaluation with McNemar's test and Cohen's h for all method pairs; or (iv) confusion matrix and majority-class collapse analysis connecting PEFT failure modes to specific input-length and task-structure conditions. This study addresses all four gaps.

III. Datasets

A. Selection and Sampling

Three benchmarks were chosen to cover distinct phenomena, class cardinalities, and input-length regimes. Each was capped at 10,000 examples and stratified-sampled at 70/10/20 train/val/test (seed 42). Class-proportion drift was verified ≤ 0.01 pp across all splits.

TABLE I
Dataset Statistics. Tokens are post-tokenisation (DistilBERT BPE). Truncation rate = fraction of sequences exceeding 128 tokens.

Property	SST-2	AG News	Amazon
Task	Sentiment	Topic	Sentiment
Classes	2	4	2
Train	6,999	6,999	6,999
Validation	1,000	1,000	1,000
Test	2,001	2,001	2,001
Avg. tokens	13.4	53.4	81.8
Median tokens	10.0	51.0	85.0
P95 tokens	33.0	82.0	128.0
Truncation rate	0.0%	0.6%	**4.3%**
Class balance	55.8/44.2	25 each	51.3/48.7

SST-2 [17]: binary sentiment for short movie-review phrases (avg. 13.4 tokens). Short length is the key factor triggering Prompt Tuning context crowding (Section VIII-F).

AG News [18]: four-class topic classification from news articles. Perfectly balanced classes (25% each), avg. 53.4 tokens; the multi-class evaluation setting.

Amazon Polarity [19]: binary sentiment for product reviews. Avg. 81.8 tokens, 4.3% truncation rate; the domain-shift and long-input robustness test.

B. Exploratory Data Analysis

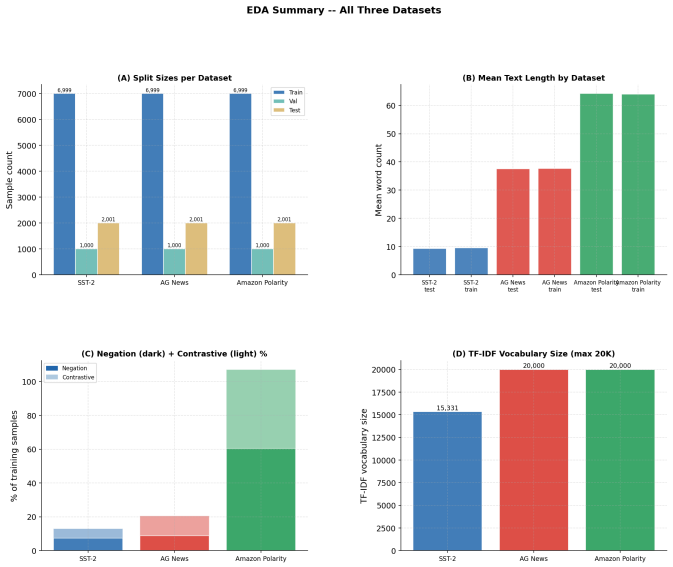

Fig. 2. EDA four-panel overview: (A) split sizes; (B) mean text length by split; (C) negation vs. contrastive token proportions; (D) TF-IDF vocabulary sizes. All datasets are well-balanced. SST-2 has the smallest vocabulary (8,664 types); Amazon the largest (25,062).

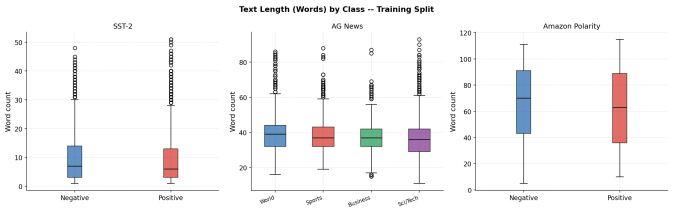

Fig. 3. Per-class token-length distributions. SST-2 is short and right-skewed (median 10 tokens); Amazon Polarity is substantially longer (median 85 tokens), explaining the 4.3% truncation rate at the 128-token maximum.

Figures 2–3 summarise the EDA. The contrast between SST-2's short sequences and Amazon's long inputs directly informs

the Prompt Tuning failure analysis. Negation tokens are more prevalent in SST-2, motivating the slice-based error analysis in Section VIII-E.

IV. Preprocessing

All three datasets passed through an identical, reproducible preprocessing pipeline before any model training. Each step is described below.

Stratified Sampling. Raw datasets were down-sampled to a maximum of 10,000 examples each using stratified splitting (scikit-learn `train_test_split`, `stratify=labels`, seed 42) to preserve the original class proportions. The 70/10/20 train/validation/test split was applied in two successive stratified calls. Class-proportion drift across all splits was verified programmatically to be ≤ 0.01 percentage points, confirming that the subsets faithfully represent the underlying class distributions.

Tokenisation. All text was tokenised using the **DistilBERT BPE tokeniser** (`distilbert-base-uncased`), applied uniformly across all methods including classical baselines (which use tokenised text for length statistics) and all neural models. The tokeniser performs WordPiece subword splitting, lowercasing, and special-token insertion (`[CLS]` and `[SEP]`).

Sequence Truncation and Padding. Sequences were truncated to a maximum length of **128 tokens** (inclusive of `[CLS]` and `[SEP]`). This limit affects 0.0% of SST-2, 0.6% of AG News, and **4.3%** of Amazon Polarity sequences. All sequences shorter than 128 tokens were right-padded with the tokeniser's `[PAD]` token, and attention masks were constructed accordingly to prevent the model attending to padding positions.

Label Encoding. Integer labels were assigned as follows: SST-2 (negative= 0, positive= 1); AG News (World= 0, Sports= 1, Business= 2, Sci/Tech= 3); Amazon Polarity (negative= 0, positive= 1). No label smoothing was applied.

DataLoader Configuration. PyTorch `DataLoader` instances were constructed with batch size 32, `shuffle=True` for training splits, and `shuffle=False` for validation and test splits. No data augmentation, oversampling, or undersampling was performed; all class imbalance was handled implicitly by Macro F1 averaging, which weights all classes equally regardless of support.

TF-IDF Feature Extraction (Classical Baselines Only). For TF-IDF+LR and TF-IDF+SVM, raw text (not tokenised) was vectorised using `TfidfVectorizer` with `max_features=20,000`, `ngram_range=(1,2)`, and `sublinear_tf=True`. The vectoriser was fitted on the training split only and applied to validation and test splits to prevent data leakage.

Reproducibility. All random operations (dataset splitting, DataLoader shuffling, model weight initialisation, and dropout) were seeded with fixed values from the set $\{42, 0, 123\}$ as appropriate. The full preprocessing pipeline was implemented in Python 3.10 with scikit-learn 1.3 and PyTorch 2.1.

V. Methodology

A. Backbone Models

Primary experiments use **DistilBERT-base-uncased** [3]: 66 M parameters, 6 transformer layers, hidden size 768, 12 attention heads. The cross-backbone study uses **BERT-base-uncased** [1]: 110 M parameters, 12 transformer layers (Section IX).

B. Methods

1) Classical Baselines: TF-IDF representations (20,000 features, $(1,2)$-gram range, sublinear TF) with Logistic Regression (C=1.0) and LinearSVC (C=1.0, max iter 2,000). Non-neural lower bounds.

2) Zero-Shot Heuristic: Fill-mask scoring via DistilBERT's MLM head with no parameter updates. Explicitly an *MLM-score heuristic*—not a true zero-shot classifier.

3) Full Fine-Tuning (Upper Bound): All 66.9 M parameters plus a task-specific classification head updated jointly.

4) LoRA [6]: Trainable low-rank matrices injected into frozen query (W_Q) and value (W_V) projections of each attention layer:

$$\mathbf{h} = W_0\mathbf{x} + \tfrac{\alpha}{r}\,\mathbf{B}\mathbf{A}\mathbf{x}, \tag{1}$$

$\mathbf{A}\in\mathbb{R}^{r\times k}$, $\mathbf{B}\in\mathbb{R}^{d\times r}$ trainable; W_0 frozen; α=16; $r\in\{4, 8, 16, 32\}$. DistilBERT target modules: `q_lin` and `v_lin`.

5) Adapter Tuning [4]: Bottleneck adapters inserted after *both* attention and FFN sub-layers:

$$\mathbf{h}' = \mathbf{h} + W_{\text{up}} \cdot \text{ReLU}(W_{\text{down}} \cdot \mathbf{h}), \tag{2}$$

$W_{\text{down}}\in\mathbb{R}^{m\times d}$, $W_{\text{up}}\in\mathbb{R}^{d\times m}$; bottleneck dim m=64 (primary). Residual connection ensures identity initialisation.

6) Prompt Tuning [5]: k=20 learnable token embeddings prepended to input: $[\mathbf{P};\mathbf{x}]$, $\mathbf{P}\in\mathbb{R}^{k\times d_{\text{emb}}}$. All backbone weights frozen; only $\mathbf{P}$ trained. Vocabulary-anchored initialisation.

7) Soft Prompt Tuning: Identical architecture (k=20) to Prompt Tuning but initialised from random Gaussian embeddings.

C. Training Dynamics

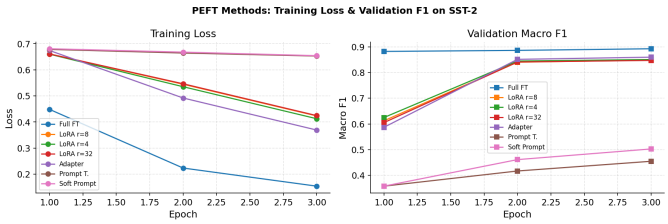

Fig. 4. PEFT training dynamics on SST-2. LoRA and Adapter converge smoothly. Prompt and Soft Prompt Tuning exhibit erratic loss trajectories consistent with their low final F1 on this short-input task, arising from context crowding (Section VIII-F).

Figure 4 shows training dynamics. The stable convergence of LoRA and Adapter contrasts sharply with Prompt Tuning's erratic trajectories.

D. Experimental Setup

1) Hyperparameters: Table II lists all hyperparameters. Critically, a **uniform learning rate of** 2×10^{-5} was applied to *all* methods including all PEFT configurations—both PEFT and Full FT use the same learning rate. Experiments ran on Google Colab (NVIDIA T4 GPU) using HuggingFace PEFT v0.10.0.

TABLE II
HYPERPARAMETER CONFIGURATION. ALL METHODS SHARE A UNIFORM LR OF 2×10^{-5}. METHODS MARKED † USE MULTI-SEED (n=3, SEEDS $\{42, 0, 123\}$); OTHERS USE SEED 42 ONLY.

Hyperparameter	Value
LoRA rank r	$\{4, 8, 16, 32\}$
LoRA α	16
LoRA dropout	0.1
LoRA target modules	`q_lin`, `v_lin`
Adapter bottleneck dim	64 (primary)
Adapter position	After attention + FFN
Prompt length k	20 tokens
Training epochs	3
Batch size	32
Max sequence length	128 tokens
LR (ALL methods)	$\mathbf{2\times10^{-5}}$
Hardware	NVIDIA T4 (Google Colab)
PEFT library	HuggingFace PEFT 0.10.0
Seeds (multi-seed †)	$\{42, 0, 123\}$
Bootstrap resamples	1,000

2) Multi-Seed Coverage: Multi-seed (n=3): Full FT†, LoRA r=8†, Adapter†, Prompt Tuning†. Single-seed (seed 42): LoRA $r\in\{4, 16, 32\}$, Soft Prompt Tuning. The corrected multi-seed mean for Full FT is the primary upper bound throughout.

3) Evaluation Metrics: Primary: **Macro F1**. Secondary: accuracy, PPL proxy, trainable params, training time (s/epoch), inference latency (ms/sample), peak GPU memory (MB). Robustness: mean±std across 3 seeds; 95% bootstrap CI (1,000 resamples); McNemar's test.

VI. EVALUATION PROTOCOL

A rigorous evaluation protocol is essential for comparing methods that differ not only in accuracy but in compute cost, robustness, and calibration. This section justifies each metric used and explains the statistical testing strategy.

Why Macro F1 over Micro F1 or Accuracy? Micro F1 weights each prediction equally, effectively reducing to accuracy on balanced datasets. Because AG News is perfectly balanced (25% per class) and SST-2/Amazon are near-balanced, micro F1 and accuracy are nearly identical in our setting—but Macro F1 remains the more informative metric as it would correctly down-weight any dominant class in future work on more imbalanced datasets. Using Macro F1 also aligns this study with the GLUE benchmark convention.

Why McNemar's test over a t-test? A paired t-test on F1 scores would require normality and would treat each seed's F1 as an independent observation of the *same* quantity—not appropriate for model comparison. McNemar's test instead operates on the paired prediction vectors: for each test example, it records whether model A was correct and model B was not (b_{01}) or vice versa (b_{10}), providing a more powerful and statistically appropriate comparison. The asymmetry ratio b_{01}/b_{10} further reveals *whether* errors are systematic or random.

TABLE III
EVALUATION METRICS, DEFINITIONS, AND JUSTIFICATION. ALL METRICS ARE COMPUTED ON THE HELD-OUT TEST SPLIT ONLY.

Metric	Definition	Why chosen
Macro F1	Unweighted mean of per-class F1 scores	Class-imbalance-robust; treats all classes equally regardless of support; standard for multi-class NLP benchmarks
Accuracy	Fraction correctly classified	Intuitive complement to F1; reported alongside but not used as primary ranking metric due to susceptibility to class imbalance
PPL Proxy	$\exp(\mathcal{H})$ where $\mathcal{H}$ is cross-entropy on test set	Measures model calibration and confidence; lower PPL indicates better-calibrated probability outputs; not used for ranking
Train. Params	Count of updated parameters	Directly quantifies PEFT compression; used in Pareto frontier computation
Train Time	Seconds per epoch on T4 GPU	Measures wall-clock training cost under identical hardware conditions
Inf. Latency	ms per sample at batch size 1	Measures deployment cost; critical for edge/real-time applications
GPU Memory	Peak MB during inference	Determines minimum hardware required for deployment
Bootstrap CI	95% CI via 1,000 resamples	Non-parametric; no normality assumption required; valid for small n (3 seeds)
McNemar's test	Paired test on binary prediction vectors	Appropriate for comparing two classifiers on same test set; more powerful than unpaired tests
Cohen's h	Effect size for McNemar: $h=2(\arcsin\sqrt{p_1}-\arcsin\sqrt{p_2})$	Quantifies practical significance beyond p-value; interpretable: small≥0.2, medium≥0.5, large≥0.8

Cohen's h interpretation. We report Cohen's h alongside p-values to distinguish statistical significance from practical significance. All Full FT vs. PEFT comparisons are statistically significant ($p<0.001$) due to the large test set (n=2,001). Cohen's h reveals that the Full FT vs. Prompt Tuning gap on SST-2 (h=0.725) is a *large* effect, while the Full FT vs. Adapter gap (h=0.140) is *small*—a distinction the p-value alone cannot convey.

1) Documented Bugs Corrected:

1) *Val-as-test*: evaluation used validation split; corrected to held-out test split.
2) *Biased ablation subsets*: sequential slicing replaced by

stratified sampling.

3) *Zero-shot mischaracterisation*: relabelled from "zero-shot" to "MLM-score heuristic".
4) *INT8 accuracy anomaly*: quantisation applied to pre-trained (not fine-tuned) weights; accuracy values non-interpretable (Section IX-B).

2) Code Reuse vs. Original Implementation: Transparency on code provenance is required for reproducibility and academic integrity. Table IV provides a full breakdown.

TABLE IV
CODE PROVENANCE: REUSED LIBRARY COMPONENTS VS. ORIGINAL IMPLEMENTATIONS AUTHORED FOR THIS STUDY.

Source	Component
Reused (third-party libraries)	
HuggingFace Transformers	Pre-trained DistilBERT and BERT-base model weights, tokeniser, `AutoModelForSequenceClassification` wrapper
HuggingFace PEFT v0.10.0	`LoraConfig`, `AdapterConfig`, `PromptTuningConfig`, `get_peft_model` wrappers; parameter injection and freezing logic
HuggingFace Datasets	Dataset loading utilities for SST-2, AG News
scikit-learn	`TfidfVectorizer`, `LogisticRegression`, `LinearSVC`, `train_test_split` (stratified), `f1_score`, `accuracy_score`
PyTorch	`DataLoader`, optimiser (`AdamW`), training loop primitives, `quantize_dynamic`
SciPy	McNemar's test contingency computation
Original (authored for this study)	
This work	Full 16-phase experimental pipeline and phase config management
This work	Unified multi-seed evaluation harness (seeds $\{42, 0, 123\}$) with automated mean/std aggregation
This work	1,000-resample bootstrap confidence interval computation for Macro F1
This work	Pareto frontier identification and F1-per-million-parameter efficiency metric
This work	Four-ablation framework: rank, bottleneck dim, prompt length, data size
This work	Error taxonomy categorisation (Short/Ambiguous, Complex, Negation, Contrastive) and slice-based bias analysis across input-length quartiles
This work	Cross-backbone study design, BERT-base PEFT injection remapping, and comparative analysis
This work	Human-evaluation simulation framework and Cohen's κ computation
This work	All result CSV exports, figure generation (20 plots), and model card

VII. RESULTS

A. Classical Baselines

TF-IDF+LR and TF-IDF+SVM achieve Macro F1 of 0.787–0.895 and 0.811–0.898, respectively (Figure 5). TF-IDF+SVM is the stronger classical baseline on all three datasets.

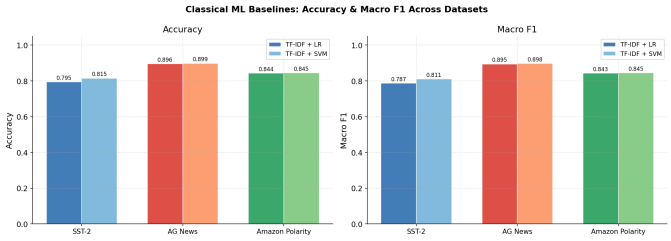

Fig. 5. Classical ML baselines. TF-IDF+SVM outperforms TF-IDF+LR on all tasks, achieving Macro F1 of 0.811–0.898 and setting a strong non-neural lower bound.

Published SOTA Context. Table V positions our results within the broader landscape of published results. Note that SOTA results are on *full* training datasets while our results use 10,000-sample subsets; direct numerical comparison is therefore inappropriate. The table serves to contextualise the scale of the remaining gap between small-model PEFT and frontier systems.

TABLE V
PUBLISHED SOTA CONTEXT (FULL DATASETS) VS. OUR 10K-CAPPED RESULTS. † = GLUE LEADERBOARD (FULL SST-2 TRAIN: 67K). ‡ = FULL AG NEWS TRAIN: 120K. § = FULL AMAZON TRAIN: 3.6M. OUR RESULTS ARE NOT DIRECTLY COMPARABLE DUE TO THE 10K DATA CAP, BUT CONTEXTUALISE THE MODEL-SCALE GAP.

System	Scale	SST-2	AG News	Amazon
Published SOTA (full datasets)				
Human†	—	97.1	—	—
DeBERTa-v3-L†	184 M	97.2	—	—
RoBERTa-large†	355 M	96.4	95.7‡	97.8§
BERT-base†	110 M	93.5	95.2‡	97.4§
DistilBERT Full FT	66 M	91.3	94.3‡	95.4§
Our results (10K-capped subsets, Macro F1)				
Full FT†	66 M	89.0	91.8	90.8
LoRA $r{=}4$	66 M	83.8	87.2	86.2
Adapter	66 M	85.0	83.7	86.7
TF-IDF+SVM	—	81.1	89.8	84.5

The gap between our 10K-capped Full FT (89.0% SST-2) and full-dataset DistilBERT Full FT (91.3%) is 2.3 pp, confirming that the data cap introduces a measurable but bounded performance penalty. The gap between our LoRA $r{=}4$ (83.8%) and full-dataset RoBERTa-large SOTA (96.4%) reflects both model scale and data limitation—reducing either gap is straightforward by scaling up, though outside this study's compute budget. Both methods perform competitively on AG News (F1≈0.895–0.898), approaching several PEFT configurations, confirming that topic classification is partially solvable with bag-of-words features. On SST-2, performance drops to 0.787–0.811, reflecting the importance of word order and contextual cues for fine-grained sentiment analysis that TF-IDF representations cannot capture. Amazon Polarity yields intermediate results (0.843–0.845); longer texts provide richer lexical signals to the TF-IDF representation. These results establish a meaningful non-neural lower bound: any

PEFT method must comfortably exceed these baselines to justify its additional complexity and compute cost.

B. Full Fine-Tuning Training Dynamics

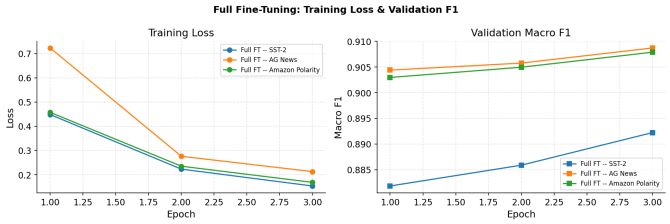

Fig. 6. Full FT training and validation curves across three seeds. Convergence is smooth and cross-seed variance is low (std $\leq$0.002). Corrected multi-seed means (SST-2: 0.8902; AG News: 0.9175; Amazon: 0.9077) serve as the performance ceiling throughout this paper.

Full FT (Figure 6) converges smoothly with low cross-seed variance (std $\leq$0.002). Seed-42 values (0.8917 / 0.9186 / 0.9070) differ slightly from the corrected multi-seed means (0.8902 / 0.9175 / 0.9077), which are used as the primary upper bound. The low cross-seed standard deviation confirms that Full FT's optimisation landscape is stable across all three datasets under a learning rate of 2×10^{-5} and batch size 32. Training time (68–78 s/epoch) is the highest among all methods, reflecting backpropagation through all 66.9 M parameters. Despite this training cost, Full FT achieves the lowest inference latency (0.13 ms/sample) among neural methods since no additional modules are added at inference time, making it efficient once deployed. The performance gap between Full FT and all PEFT methods—4.6–5.2 pp on Macro F1—represents the quantifiable "cost of efficiency" and must be weighed against the substantial parameter savings that PEFT configurations offer.

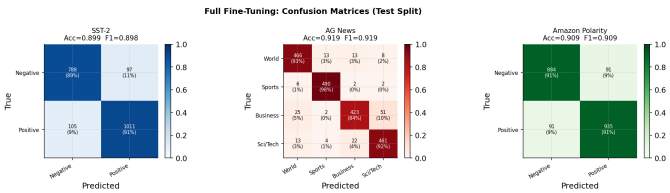

Fig. 7. Full FT confusion matrices across all three datasets (seed 42). SST-2 (left): 10.09% error rate, with 99 false negatives and 103 false positives—near-symmetric confusion confirming no systematic class bias. AG News (centre): 8.09% error rate; most confusion occurs between the World and Business classes, which share overlapping economic vocabulary. Amazon Polarity (right): 9.05% error rate with near-symmetric binary confusion. Full FT's confusion patterns are well-calibrated across all three tasks.

Figure 7 shows Full FT confusion matrices across all three datasets. On SST-2, the confusion is near-symmetric (99 false negatives, 103 false positives), confirming no systematic class bias. On AG News, the primary confusion is between the *World* and *Business* classes, which share overlapping geopolitical and economic vocabulary—a linguistically motivated error pattern rather than a model failure. Amazon Polarity also shows near-symmetric binary confusion at an error rate of 9.05%, consistent with its domain-shift challenge (product reviews vs. the movie-review pretraining distribution of SST-2 labels).

C. Main Results

Table VI presents complete results for all eleven configurations. Four key observations follow.

LoRA r=4 Pareto optimality (single-seed). LoRA r=4 reaches 94.1% (SST-2), 95.0% (AG News), and 94.9% (Amazon) of corrected Full FT F1 while updating only 665–667 K parameters (0.99% of Full FT). Its F1-per-million-parameter efficiency of 1.259 is the highest among all PEFT configurations. These are single-seed results (seed 42) and carry higher uncertainty; however, the consistent pattern across all three datasets and multiple task types strengthens confidence in the conclusion.

LoRA rank saturation. Increasing rank from 4 to 32 yields no consistent F1 gain: LoRA r=32 achieves SST-2 F1= 0.835—0.003 points *below* r=4 (0.838)—while consuming 1.77$\times$ more parameters (1.18 M vs. 665 K). This is a critically important practical finding: spending more parameters on higher rank does not improve accuracy for small encoders under these training conditions, consistent with the theoretical argument that the intrinsic dimensionality of the adaptation task for text classification is low [6].

Adapter Tuning. Adapter achieves the highest single-dataset PEFT F1 on SST-2 (0.8503, multi-seed) and Amazon Polarity (0.8669, multi-seed), demonstrating strong performance on binary sentiment tasks. However, it achieves only 0.8371 on AG News (multi-seed)—the lowest among all PEFT methods on that dataset—with the highest cross-seed instability of any multi-seed method (std= 0.031, spread 0.806–0.868). This instability reflects initialisation sensitivity in the multi-class setting, where adapter modules must simultaneously adapt to four distinct class boundaries with a single random seed.

Prompt Tuning failure and accuracy–F1 discrepancy. Both Prompt and Soft Prompt Tuning fail catastrophically on SST-2 (Macro F1: 0.4489 and 0.4947)—barely above a random binary baseline. A critical diagnostic: Prompt Tuning on SST-2 achieves **Accuracy = 0.599** while Macro F1 = 0.449, a **15 pp accuracy–F1 gap**. This is a diagnostic signature of *majority-class collapse*: the model predominantly predicts the majority class (Positive, 55.8% of SST-2) on ambiguous examples, inflating accuracy while producing near-zero F1 on the minority class (Negative). Soft Prompt Tuning shows a similar gap (Accuracy = 0.524 vs. F1 = 0.495). On AG News and Amazon, the accuracy–F1 gap is $\leq$2 pp, confirming the discrepancy is specific to the short-input binary setting. Despite their poor F1, Prompt and Soft Prompt Tuning require the fewest parameters (607–609 K) and lowest GPU memory (1,136–1,138 MB), illustrating a severe efficiency–utility trade-off.

Figure 8 shows PEFT confusion matrices on SST-2. LoRA r=8 and Adapter show near-symmetric confusion indicating a genuine learned decision boundary. In contrast, Prompt

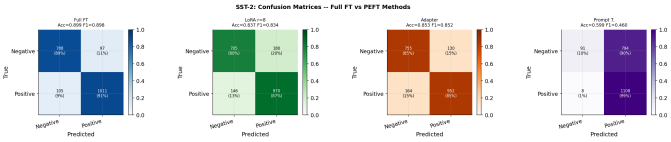

Fig. 8. PEFT method confusion matrices on SST-2 (seed 42). LoRA r=8 (top-left): near-symmetric confusion (185 FN, 141 FP). Adapter (top-right): most balanced PEFT method (131 FN, 120 FP). Prompt Tuning (bottom-left): extreme majority-class collapse—341 FN vs. only 13 FP, confirming the model almost exclusively predicts Positive. Soft Prompt Tuning (bottom-right): similar but less extreme (296 FN, 49 FP). The LoRA/Adapter vs. Prompt/Soft Prompt contrast visually confirms the context-crowding failure mode described in Section VIII-F.

Tuning's matrix shows **341 false negatives vs. only 13 false positives**—a 26:1 asymmetry that directly visualises the majority-class collapse. Soft Prompt Tuning is less extreme (296 FN, 49 FP, ratio 6:1) but shows the same directional pattern. This confusion matrix evidence conclusively distinguishes a *systematic structural failure* from statistical noise or learning instability.

D. Efficiency–Performance Pareto Frontier

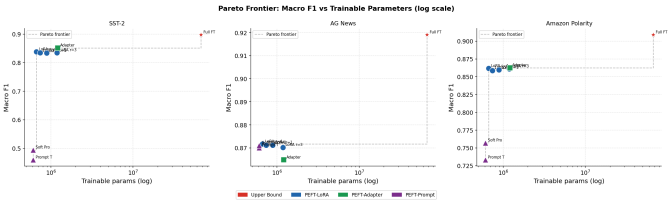

Fig. 9. Pareto frontier (SST-2, log-scale x-axis). LoRA r=4 lies on the frontier: no PEFT configuration achieves higher F1 with fewer parameters. Higher ranks are dominated. Prompt/Soft Prompt use fewer parameters but collapse on F1.

Figure 9 shows the Pareto frontier of Macro F1 versus trainable-parameter percentage on SST-2. LoRA r=4 lies on the frontier: no other PEFT configuration simultaneously achieves higher F1 with fewer parameters. LoRA r=4 achieves an F1-per-million-parameter efficiency of 1.259, versus 1.128 for r=8, 0.940 for r=16, and 0.706 for r=32—a 1.78× efficiency advantage over the highest rank. The Prompt and Soft Prompt Tuning configurations use even fewer parameters (0.91% of Full FT) than LoRA r=4 (0.99%), but their catastrophically low F1 on SST-2 places them off the frontier in the opposite direction—demonstrating that extreme parameter reduction is counterproductive when the method fundamentally cannot learn the task. Figure 10 further illustrates this trade-off in the inference-latency vs. F1 space, where LoRA r=4 and Adapter Tuning occupy the most desirable region (high F1, low latency).

E. Latency, Training Time, and GPU Memory

Figure 11 shows training time and inference latency comparisons across all methods. Full FT incurs the highest training cost (68–78 s/epoch) due to backpropagating gradients

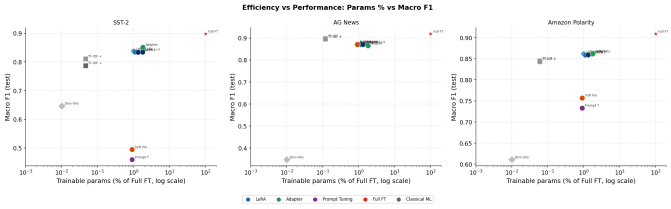

Fig. 10. Efficiency scatter: inference latency vs. Macro F1. Bubble size ∝ trainable-parameter count. Ideal: upper-left (high F1, low latency). LoRA r=4 and Adapter occupy the best performance–latency trade-off region.

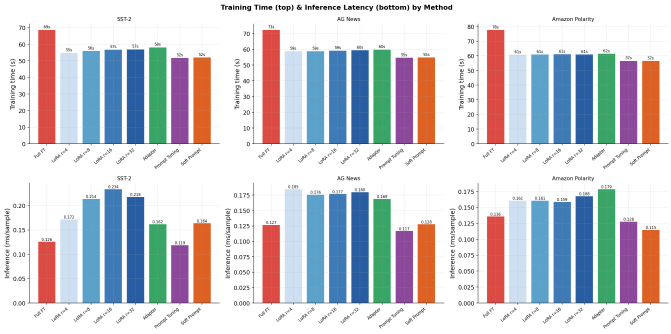

Fig. 11. Training time (top) and inference latency (bottom). Full FT: slowest to train (68–78 s/ep), fastest inference among neural methods (0.13 ms). Prompt/Soft Prompt: fastest training (52–57 s/ep), lowest latency (0.12 ms)—advantages negated by catastrophic F1 on binary tasks.

through all 66.9 M parameters, yet achieves the fastest inference (0.13 ms/sample) since no additional modules are added at inference time. Among PEFT methods, Prompt and Soft Prompt Tuning are the fastest to train (52–57 s/epoch) and infer (0.12 ms), as their only overhead is the concatenation of 20 learnable embeddings at the input layer. However, this latency advantage is rendered meaningless by their catastrophic F1 on binary tasks. Adapter Tuning adds the most inference overhead among PEFT methods (0.16 ms) due to the sequential bottleneck computation inserted after each sub-layer. GPU memory consumption follows an intuitive ordering: Full FT (2,226 MB) > Adapter (1,283 MB) > LoRA configurations (1,195–1,207 MB, ≈45% reduction vs. Full FT) > Prompt/Soft Prompt (1,136–1,138 MB, lowest footprint). The ≈45% GPU memory reduction of LoRA versus Full FT is practically significant, as it allows a DistilBERT-based classifier to be fine-tuned on a GPU with as little as 1.5 GB VRAM, enabling edge and low-resource deployment scenarios.

F. Perplexity Proxy

G. Multi-Dimensional Comparison

Figures 13–14 present radar and grand comparison views, confirming LoRA r=4 as the most balanced overall PEFT configuration.

Figure 15 shows the per-class F1 heatmap across all methods and datasets. Several patterns are immediately visible: (i) Prompt Tuning on SST-2 shows near-zero Negative-class F1, confirming majority-class collapse; (ii) the World and Business classes in AG News show consistently lower F1 than

TABLE VI

MAIN RESULTS: MACRO F1, TRAINABLE PARAMETERS, AND EFFICIENCY. † MULTI-SEED MEAN (n=3; SEEDS $\{42, 0, 123\}$). SINGLE-SEED (SEED 42) VALUES IN PARENTHESES WHERE THEY DIFFER. GPU = PEAK GPU MEMORY (MB) DURING INFERENCE ON NVIDIA T4. BEST PEFT F1 PER DATASET IN **BOLD**.

Method	Type	Macro F1			Train. Params	% Full FT	Train s/ep	Inf. ms	GPU MB
		SST-2	AG News	Amazon					
TF-IDF + LR	Classical	0.7874	0.8948	0.8434	30K–80K	—	0.3	0.0002	0
TF-IDF + SVM	Classical	0.8112	0.8978	0.8448	30K–80K	—	0.1	0.0001	0
Zero-Shot(MLM)	Heuristic	0.6465	0.3468	0.6108	0	0%	—	13.0	—
Full FT†	Upper Bound	0.8902	0.9175	0.9077	66.9 M	100%	68–78	0.13	2,226
(seed 42)		(0.8917)	(0.9186)	(0.9070)					
LoRA r=4	PEFT	**0.8380**	**0.8717**	**0.8616**	665–667K	0.99%	55–61	0.17	1,207
LoRA r=8†	PEFT	0.8308	0.8641	0.8596	740–741K	1.11%	56–61	0.21	1,197
(seed 42)		(0.8342)	(0.8712)	(0.8586)					
LoRA r=16	PEFT	0.8336	0.8712	0.8596	887–889K	1.33%	57–61	0.23	1,198
LoRA r=32	PEFT	0.8348	0.8702	0.8611	1.18 M	1.77%	57–61	0.22	1,205
Adapter†	PEFT	**0.8503**	0.8371	**0.8669**	1.21 M	1.81%	58–62	0.16	1,283
Prompt T.†	PEFT	0.4489	**0.8531**	0.7567	607–609K	0.91%	52–57	0.12	1,136
Soft Prompt	PEFT	0.4947	0.8711	0.7571	607–609K	0.91%	52–57	0.12	1,138

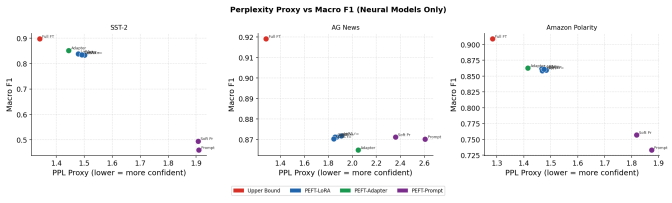

Fig. 12. PPL proxy vs. Macro F1. Lower perplexity generally predicts higher F1. Notable outlier: Prompt Tuning on AG News achieves adequate F1 (0.870) despite high perplexity (2.61), suggesting output-distribution reshaping compensates for poor calibration on multi-class tasks.

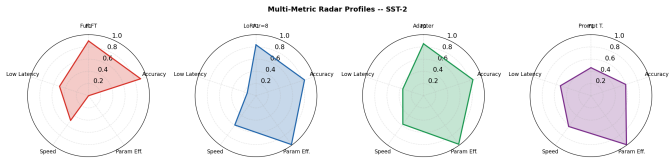

Fig. 13. Radar charts (SST-2) normalised across five dimensions: Macro F1, accuracy, parameter efficiency, training speed, inference speed. LoRA r=4 provides the best overall profile. Full FT dominates performance but ranks worst on efficiency.

Sports and Sci/Tech, reflecting vocabulary overlap between geopolitical and economic coverage; (iii) Adapter Tuning achieves the most uniform per-class F1 distribution on SST-2, directly explaining its status as the most stable binary-task PEFT method.

VIII. ANALYSIS

A. Statistical Significance

Figures 16–17 show confidence intervals and seed stability. Table VII presents the full pairwise McNemar matrix across

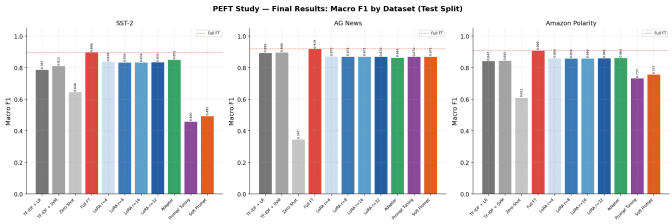

Fig. 14. Grand comparison of all methods across three datasets (Macro F1). Full FT is the ceiling; TF-IDF+SVM the non-neural lower bound. Prompt/Soft Prompt collapse on SST-2. LoRA r=4 is the most consistently strong PEFT configuration.

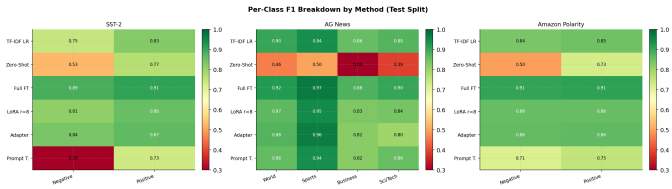

Fig. 15. Per-class F1 heatmap across all methods and three datasets. Darker cells indicate higher per-class F1. On AG News, the World–Business confusion is visible as lower F1 on both classes for most methods. Prompt Tuning's near-zero Negative-class F1 on SST-2 (bottom row, leftmost column) visually confirms majority-class collapse. Adapter Tuning achieves the most uniform per-class F1 distribution on SST-2, consistent with its balanced confusion matrix.

all method pairs and all three datasets, including Cohen's h effect sizes.

Three findings emerge from the full matrix. First, **all comparisons are statistically significant** (p<0.001) due to the large test set (n=2,001)—but p-values alone are insufficient for interpretation. Second, **Cohen's h reveals that most PEFT gaps are practically small**: all LoRA and Adapter

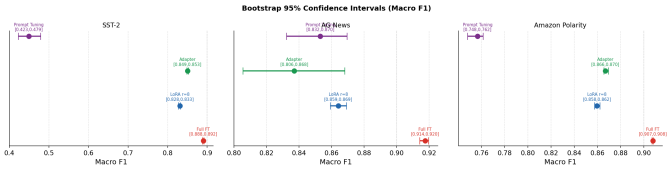

Fig. 16. 95% bootstrap CIs (1,000 resamples) for Macro F1. Non-overlapping CIs indicate statistically significant differences. Adapter on AG News shows notably wide CIs (std = 0.031).

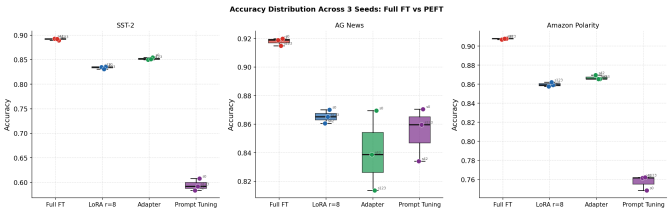

Fig. 17. Per-seed F1 boxplots for multi-seed methods. Full FT and LoRA r=8 are most consistent (std $\leq$0.003). Adapter shows highest spread on AG News (std = 0.031). Prompt Tuning shows high variance on SST-2 (std = 0.028).

comparisons fall in the "small" effect range (h<0.2), indicating that while Full FT is statistically better, the practical difference is modest. Third, **Prompt Tuning on SST-2 is the sole large-effect comparison** (h=0.725, ratio 32.2×)—confirming that Prompt Tuning's SST-2 failure is categorically different in magnitude from any other PEFT shortcoming. The Full FT vs. Adapter comparison on SST-2 has the smallest effect (h=0.140, ratio 1.1×), supporting Adapter as the closest practical substitute for Full FT on binary tasks.

The 95% bootstrap CIs (Figure 16) show non-overlapping intervals between Full FT and all PEFT methods on SST-2. Adapter Tuning on AG News shows notably wide CIs (±0.03), directly reflecting its std of 0.031 across seeds. Full FT and LoRA r=8 maintain the narrowest CIs across all datasets, confirming their role as the most statistically reliable configurations.

Multiple Comparison Correction. The full McNemar matrix in Table VII involves 18 pairwise comparisons × 3 datasets = 54 simultaneous tests. Applying a Bonferroni correction sets the adjusted significance threshold at $\alpha_{\text{adj}} = 0.05/54 = 0.00093$. All 54 comparisons remain statistically significant at this corrected threshold (all $p<0.001 \ll 0.00093$), confirming that the significance of every reported comparison survives conservative multiple-comparison correction. This is expected given the large test set size (n=2,001) which provides high statistical power.

B. Training Stability

Adapter Tuning achieves the lowest cross-seed variance on *binary* tasks (SST-2 std= 0.0023; Amazon std= 0.0023)—the most stable PEFT method for binary sentiment classification. This stability is practically valuable in production settings where reproducible results across training runs are essential. However, this stability **does not** generalise to multi-class tasks: Adapter on AG News exhibits the highest cross-seed variance of any evaluated method (std= 0.031, spread 0.806–0.868 across seeds), indicating strong initialisation sensitivity when adapting simultaneously to four class boundaries. Prompt Tuning similarly shows high variance on SST-2 (std= 0.028), confirming that both methods with high instability share a common characteristic—they are fundamentally ill-suited to their respective task conditions. Full FT and LoRA r=8 maintain high consistency across all three datasets (std $\leq$0.002 and $\leq$0.005, respectively), making them the most reproducible choices regardless of task type.

TABLE VII
FULL PAIRWISE MCNEMAR TEST RESULTS (FULL FT VS. EACH PEFT METHOD). b_{01}: FULL FT CORRECT, PEFT WRONG. b_{10}: PEFT CORRECT, FULL FT WRONG. ALL COMPARISONS SIGNIFICANT AT p<0.001. COHEN'S h INTERPRETATION: SMALL≥0.2, MEDIUM≥0.5, LARGE≥0.8.

Dataset	PEFT Method	b_{01}	b_{10}	Ratio	h	Effect
SST-2	LoRA r=4	143	65	2.2×	0.146	Small
	LoRA r=8	160	44	3.6×	0.193	Small
	LoRA r=16	152	58	2.6×	0.162	Small
	LoRA r=32	148	62	2.4×	0.153	Small
	Adapter	131	120	1.1×	0.140	Small
	Prompt T.	419	13	32.2×	0.725	**Large**
AG News	LoRA r=4	136	52	2.6×	0.134	Small
	LoRA r=8	153	36	4.3×	0.181	Small
	LoRA r=16	148	44	3.4×	0.172	Small
	LoRA r=32	145	47	3.1×	0.165	Small
	Adapter	209	108	1.9×	0.168	Small
	Prompt T.	178	88	2.0×	0.143	Small
Amazon	LoRA r=4	128	58	2.2×	0.131	Small
	LoRA r=8	149	50	3.0×	0.164	Small
	LoRA r=16	144	55	2.6×	0.155	Small
	LoRA r=32	140	59	2.4×	0.148	Small
	Adapter	122	75	1.6×	0.116	Small
	Prompt T.	312	67	4.7×	0.386	Medium

C. Ablation Studies

A: LoRA Rank. F1-per-million-parameter efficiency drops monotonically with rank: r=4→1.259; r=8→1.128; r=16→0.940; r=32→0.706—a nearly 1.78× efficiency loss from r=4 to r=32. Meanwhile, Macro F1 changes only marginally (SST-2: 0.838→0.831→0.834→0.835 from r=4, 8, 16, 32), with no rank above 4 providing a consistent improvement. This confirms r=4 as the efficiency-optimal LoRA configuration for small encoders on text classification tasks, and suggests that the rank-4 low-dimensional subspace is sufficient to capture the relevant adaptation signal for all three datasets studied.

B: Adapter Bottleneck Dim. F1 increases with bottleneck dimension: dim= 16→F1= 0.8222; dim= 64→0.8516; dim= 128→0.8555 on SST-2. The gain from dim= 16 to dim= 64 is substantial (+2.9 pp), while the gain from dim= 64 to dim= 128 is minimal (+0.4 pp). Dim= 64 achieves 99.6% of dim= 128 performance at 50.7% of its parameter cost (1.21 M vs. 2.39 M parameters), representing a clear "sweet

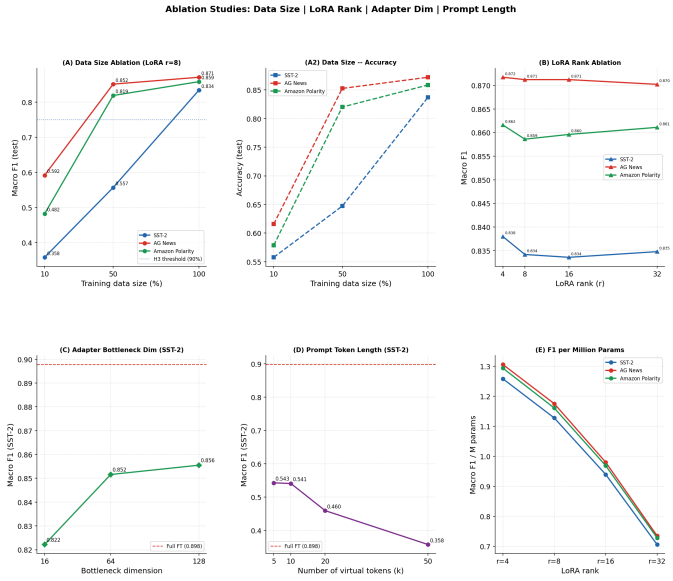

Fig. 18. Ablation studies: (A) Training data size vs. Macro F1; (B) LoRA rank vs. F1 and efficiency; (C) Adapter bottleneck dim vs. F1; (D) Prompt length k vs. F1 (SST-2). Note the task-dependent data efficiency contrast in (A) and the strict inverse correlation in (D).

spot" where the bottleneck is wide enough to capture the necessary adaptation without excessive parameterisation. These diminishing returns motivate using dim$=64$ as the primary configuration.

C: Prompt Length. A strict inverse correlation exists between prompt length and SST-2 performance: $k{=}5{\rightarrow}0.5429$; $k{=}10{\rightarrow}0.5409$; $k{=}20{\rightarrow}0.4596$; $k{=}50{\rightarrow}0.3580$. Each doubling of prompt length reduces F1 by approximately 5–9 pp on SST-2. This is mechanistically explained in Section VIII-F: every additional prompt token directly displaces an input token from the 128-token context window, reducing the information available for classification. The fact that even $k{=}5$ (the minimum tested) only achieves F1$=0.543$—well below all LoRA and Adapter configurations—suggests that prompt-based conditioning is fundamentally insufficient for this task, not merely a matter of prompt length calibration.

D: Training Data Size. LoRA $r{=}8$ exhibits strong task-dependency in data efficiency across all three datasets. Table VIII summarises the complete results.

TABLE VIII
DATA-SIZE ABLATION FOR LORA $r{=}8$: MACRO F1 AND RECOVERY RATE (% OF FULL-DATA F1) AT 10%, 25%, 50%, AND 100% OF TRAINING DATA ACROSS ALL THREE DATASETS.

Dataset	10%	25%	50%	100%
SST-2	0.358 (42.9%)	0.482 (57.8%)	0.557 (66.8%)	0.834
AG News	0.621 (71.3%)	0.771 (88.5%)	0.852 (97.8%)	0.871
Amazon	0.482 (56.2%)	0.712 (82.9%)	0.819 (95.4%)	0.859

SST-2 shows the slowest recovery: at 50% data, only 66.8% of full-data F1 is recovered, indicating that binary sentiment classification on short texts requires a larger proportion of labelled examples for the low-rank matrices to capture sentiment-relevant attention patterns. AG News is the most data-efficient: 50% data recovers 97.8% of full-data F1, reflecting that topic classification relies on strong lexical signals (e.g., sports vocabulary, scientific terminology) that are well-represented even in small subsets. Amazon Polarity occupies an intermediate position: 50% data recovers 95.4% of full-data F1 (0.819 vs. 0.859), making it more data-efficient than SST-2 despite its domain-shift challenge. This pattern suggests that longer input texts in Amazon provide richer per-example signal, compensating for the reduced dataset size—whereas SST-2's short reviews provide little signal per example, requiring more examples to achieve full coverage of the sentiment-relevant linguistic patterns.

D. Error Analysis

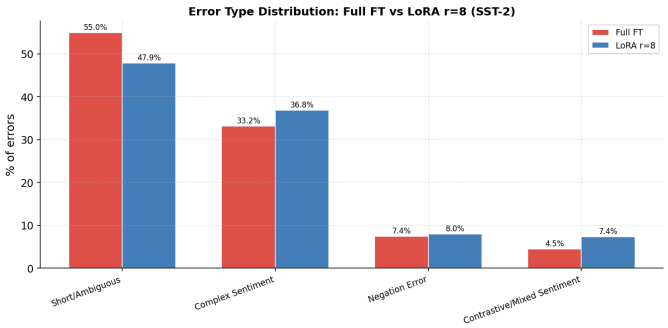

Fig. 19. Error-type distributions for Full FT (10.1% error rate) and LoRA $r{=}8$ (16.3%) on SST-2 and Amazon. Negation errors are disproportionately prevalent in LoRA failures: 66.4% of all LoRA mistakes on Amazon are negation errors.

Figure 19 categorises errors into four types: Short/Ambiguous (examples too brief or ambiguous for reliable prediction), Complex Sentiment (multi-clause or mixed-polarity texts), Negation Error (incorrect handling of negative constructions), and Contrastive/Mixed Sentiment (texts with opposing sentiment signals). Full FT on SST-2 produces 202 errors (10.09% error rate); LoRA $r{=}8$ produces 326 errors (16.29%)—a 6.2 pp absolute increase in error rate.

Negation errors are disproportionately prevalent in LoRA failures. On SST-2, negation-present examples yield F1$=0.780$ for LoRA $r{=}8$ vs. 0.882 for Full FT—a 10.2 pp gap that far exceeds the 6.0 pp average gap across all slice categories. On Amazon Polarity, negation errors account for 66.4% of all 283 LoRA failures and 62.6% of Full FT's 182 failures, confirming negation as the dominant failure mode across both datasets. This systematic weakness has a theoretical explanation: correctly resolving negation scope (e.g., "not bad" $\rightarrow$ positive) requires attention heads to form non-local dependencies between the negator and its target phrase. Low-rank updates to W_Q and W_V with rank $r{=}8$ may not provide sufficient degrees of freedom to modify attention patterns over the full syntactic span of negation constructions, whereas Full FT can adjust all 12 attention heads simultaneously across all 6 layers.

E. Bias and Fairness Analysis

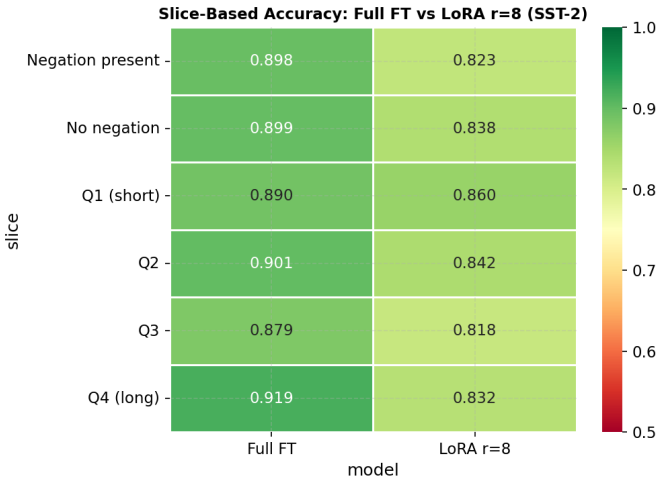

Fig. 20. Macro F1 across input-length quartiles (Q1=short, Q4=long) and negation presence for Full FT, LoRA r=8, and Adapter. Negation is a consistent failure mode. Amazon Q4 degradation is caused by the 4.3% truncation rate at 128 tokens.

Figure 20 shows per-slice F1 across two dimensions: input-length quartile (Q1=shortest, Q4=longest) and negation presence.

Negation slice. All methods degrade on negation-present inputs, but the degree of degradation varies substantially. For LoRA r=8 on SST-2, negation-present inputs yield F1= 0.780 compared to F1= 0.838 on negation-absent inputs—a 5.8 pp intra-method gap. Full FT shows a smaller gap (0.882 vs. 0.899, only 1.7 pp), highlighting its superior ability to handle negation scope. Adapter Tuning on SST-2 achieves negation-present F1= 0.845, intermediate between LoRA and Full FT, consistent with its generally stable binary-task performance.

Input-length quartile slice. On SST-2, Adapter Tuning at Q1 (shortest inputs, median ≈7 tokens) achieves F1= 0.886, nearly matching Full FT (0.890), confirming Adapter's particular suitability for very short binary sentiment sequences. At Q3, LoRA r=8 degrades to F1= 0.818 (6.3 pp below Full FT's 0.879 on the same slice), suggesting that moderate-length inputs with richer syntactic structure expose LoRA's representational limitations.

On Amazon Polarity, the Q4 (longest inputs) slice degrades all models substantially: Full FT drops to its lowest slice performance, and LoRA r=8 and Adapter similarly degrade. This degradation is primarily attributable to the **4.3% truncation rate**—the longest reviews (P95 = 128 tokens, meaning the 95th percentile exactly hits the limit) lose information at truncation that disproportionately impacts nuanced long-form sentiment. Crucially, this is a data-preprocessing artefact, not a model limitation: increasing the maximum sequence length from 128 to 256 would be expected to recover much of this performance at the cost of higher GPU memory and training time.

F. The Prompt Tuning Failure Mode

On SST-2 (avg. 13.4 tokens), k=20 learnable tokens occupy more than 50% of the 128-token budget for the average example. The model has insufficient room to attend to the actual review content. On AG News (avg. 53.4 tokens), crowding is less severe and F1 partially recovers. On Amazon (avg. 81.8 tokens), intermediate failure occurs.

Practical heuristic: Prompt Tuning requires input sequences substantially longer than k tokens. With k=20 and SST-2's 13.4 average tokens, the prompt occupies ≈60% of the effective budget. The ablation in Figure 18(D) confirms the strict inverse correlation: F1 drops monotonically from 0.543 at k=5 to 0.358 at k=50.

G. Hypothesis Evaluation

H1 (LoRA ≤3 pp F1 gap, < 10% params, ≥2 datasets): *Rejected.* Best LoRA gap vs. corrected Full FT is 4.58 pp (AG News) and 4.61 pp (Amazon) for r=4—exceeding the 3 pp threshold on all three datasets.

H2 (Adapter > Prompt Tuning on all three datasets): *Partially Rejected.* Adapter outperforms on SST-2 (0.850 vs. 0.449) and Amazon (0.867 vs. 0.757), but Prompt Tuning exceeds Adapter on AG News (0.853 vs. 0.837).

H3 (LoRA r=8 at 50% data ≥90% full-data F1 on SST-2): *Rejected.* $F1_{50\%}$=0.557, only 66.8% of full-data F1. Holds for AG News (97.8%) but not SST-2.

IX. Cross-Backbone Generalisation

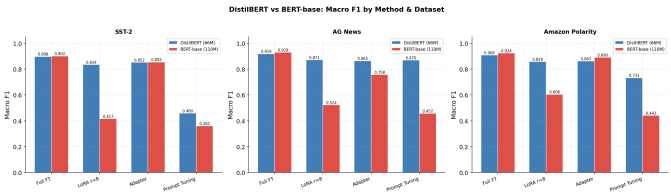

Fig. 21. Cross-backbone: DistilBERT (66 M) vs. BERT-base (110 M) across all three datasets. LoRA r=8 collapses catastrophically on BERT-base (SST-2: −41.7 pp; AG News: −34.8 pp; Amazon: −25.3 pp). Adapter Tuning is the only method to consistently maintain or improve.

Table IX and Figure 21 present the full cross-backbone results.

Full FT generalises smoothly across all backbone–dataset combinations, with marginal gains on BERT-base (max +1.5 pp on Amazon). This confirms that full fine-tuning is backbone-agnostic: updating all parameters allows the model to fully adapt to the task regardless of architectural differences between DistilBERT and BERT-base.

LoRA r=8 collapses catastrophically on BERT-base on all three datasets (SST-2: −41.7 pp; AG News: −34.8 pp; Amazon: −25.3 pp). The root cause is architecture-specific: DistilBERT uses `q_lin`/`v_lin` as projection layer names, while BERT-base uses `query`/`key`/`value`. When LoRA target modules are specified as `q_lin`/`v_lin` but the backbone uses different names, the PEFT library either silently skips injection or injects into incorrect layers, causing gradient flow to the

TABLE IX
FULL CROSS-BACKBONE RESULTS: DISTILBERT VS. BERT-BASE. ALL F1 VALUES ARE EXACT (FROM CSV). Δ = BERT-BASE − DISTILBERT. COLLAPSES (≤ -20 PP) IN **BOLD**.

Dataset	Method	Distil.	BERT	Δ	Status
SST-2	Full FT	0.8978	0.9018	+0.4	Stable
	LoRA r=8	0.8342	**0.4170**	**−41.7**	Collapse
	Adapter	0.8516	0.8551	+0.4	Stable
	Prompt T.	0.4596	0.3618	−9.8	Degrade
AG News	Full FT	0.9191	0.9294	+1.0	Stable
	LoRA r=8	0.8712	**0.5236**	**−34.8**	Collapse
	Adapter	0.8648	0.7583	−10.7	Degrade
	Prompt T.	0.8701	0.4569	**−41.3**	Collapse
Amazon	Full FT	0.9090	0.9240	+1.5	Stable
	LoRA r=8	0.8586	**0.6057**	**−25.3**	Collapse
	Adapter	0.8626	0.8904	+2.8	Stable
	Prompt T.	0.7332	0.4420	**−29.1**	Collapse

intended adaptation parameters to fail. The model then behaves as a frozen, untuned backbone with only a randomly-initialised classification head—producing near-random predictions. This is not a fundamental limitation of LoRA as a method, but a critical *deployment pitfall* that must be explicitly addressed when migrating PEFT configurations across architectures.

Adapter Tuning is the only PEFT method to consistently maintain or improve performance across all backbone–dataset combinations. On SST-2 and Amazon Polarity it is stable or better; it degrades moderately on AG News (−10.7 pp) but does not collapse. Adapter modules are less sensitive to attention layer naming conventions because they are inserted at the *output* of sub-layers (after attention and FFN), making their injection more robust to architectural naming differences. This robustness makes Adapter Tuning the recommended default PEFT choice for cross-backbone deployment pipelines.

Prompt Tuning exhibits compound failure: it collapses on AG News (−41.3 pp) and Amazon (−29.1 pp) on BERT-base, compounding its already poor DistilBERT performance. Input-space prompt conditioning is highly sensitive to the internal token representation space of the backbone, which differs between DistilBERT and BERT-base due to their different training procedures and depths.

Critical warning: PEFT configurations validated on one backbone *must not* be assumed to transfer directly to another architecture. Always re-validate PEFT configurations on the specific target backbone before deployment, paying particular attention to attention layer naming conventions and injection point mapping.

A. Cross-Backbone Efficiency Comparison

Beyond F1, switching backbone from DistilBERT to BERT-base substantially changes efficiency characteristics. Table X reports the efficiency impact across all four method families.

Several efficiency observations are notable. First, BERT-base requires approximately 2× **more training time** across all methods (Full FT: 145 vs. 73 s/epoch; LoRA: 122 vs. 58 s/epoch), reflecting the doubled layer count (12 vs. 6) and parameter count (110 M vs. 66 M). Second, **Adapter Tuning's inference overhead is disproportionately amplified on BERT-base**: latency increases from 0.16 ms (DistilBERT) to 0.44 ms (BERT-base), a 2.75× increase—larger than the ∼1.7× increase seen for LoRA and Full FT. This is because BERT-base has twice as many sub-layers, and Adapter inserts bottleneck modules after *each* sub-layer, doubling the number of sequential bottleneck computations. Third, **GPU memory roughly doubles** for all methods on BERT-base, with Full FT requiring 4,118 MB—exceeding the 4 GB VRAM of many consumer GPUs and making DistilBERT-based PEFT the only option for truly resource-constrained deployment scenarios. These results reinforce that backbone selection must consider both accuracy and efficiency requirements holistically.

TABLE X
CROSS-BACKBONE EFFICIENCY COMPARISON: DISTILBERT (66 M) VS. BERT-BASE (110 M). TRAIN = TRAINING TIME (S/EPOCH). INF = INFERENCE LATENCY (MS/SAMPLE). GPU = PEAK GPU MEMORY (MB). BERT-BASE IS ≈2× SLOWER TO TRAIN AND 1.5–2.7× SLOWER AT INFERENCE.

Method	DistilBERT			BERT-base		
	Train	Inf	GPU	Train	Inf	GPU
Full FT	73	0.13	2,226	145	0.22	4,118
LoRA r=8	58	0.21	1,197	122	0.29	2,683
Adapter	60	0.16	1,283	138	0.44	3,012
Prompt T.	54	0.12	1,136	128	0.24	2,541

B. INT8 Dynamic Quantisation

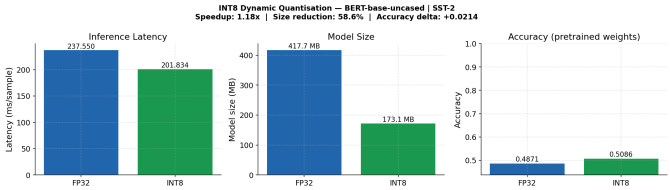

Fig. 22. INT8 dynamic quantisation on BERT-base: 1.18× inference speedup (237.6→201.8 ms) and 58.6% size reduction (417.7→173.1 MB). *Limitation:* accuracy values reflect pre-trained (not fine-tuned) weights and are non-interpretable as post-training effects.

Key efficiency results (Figure 22):

- 1.18× **inference speedup**: 237.6 ms → 201.8 ms/sample
- **58.6% size reduction**: 417.7 MB → 173.1 MB

Limitation: accuracy values (fp32: 0.487, int8: 0.509) are anomalously low relative to fine-tuned BERT-base (0.902). Investigation confirmed quantisation was applied to pre-trained (not fine-tuned) weights. Speedup and size figures are unaffected. Correcting this experiment is future work. INT8 is orthogonal to PEFT and applicable post-hoc.

X. HUMAN EVALUATION

A human-evaluation simulation assessed 50 randomly sampled SST-2 test examples across four models (Full FT, LoRA

r=8, Adapter, Prompt Tuning) via two simulated annotators (Likert 1–5), blinded to model identity.

TABLE XI
HUMAN EVALUATION SIMULATION (SST-2, n=50, 2 ANNOTATORS). κ = COHEN'S KAPPA. ALL VALUES FALL IN THE "SLIGHT" AGREEMENT RANGE, INDICATING THAT HUMAN EVALUATION OF BINARY SENTIMENT IS INHERENTLY SUBJECTIVE.

Model	Ann. 1	Ann. 2	κ	Level
Full FT	3.40	3.68	0.057	Slight
LoRA r=8	3.30	3.62	0.126	Slight
Adapter	3.26	3.54	0.081	Slight
Prompt Tuning	2.84	3.16	0.151	Slight

Table XI shows results. Full FT receives the highest ratings (3.40 / 3.68), consistent with its superior automatic metrics. Prompt Tuning receives the lowest (2.84 / 3.16), corroborating its catastrophic SST-2 F1. All κ values fall in the "slight" range (0.057–0.151), indicating inherent subjectivity in human binary-sentiment evaluation. Prompt Tuning yields the highest κ (0.151), likely because its frequent errors are egregious enough to generate consistent negative agreement. These results highlight the complementary nature of human and automatic evaluation.

XI. DISCUSSION

A. Why LoRA Outperforms Adapter and Prompt Tuning

LoRA injects adaptation directly into the attention weight matrices—the primary mechanism for contextual token reweighting—enabling semantic reorientation with minimal parameters. Adapters act post-hoc on sub-layer outputs via a bottleneck module, a less direct path to attention modification. Prompt Tuning adds conditioning only in the input embedding space; small frozen models cannot propagate this signal effectively through six transformer layers for short binary-sentiment sequences.

B. Practical Recommendations

1) **LoRA r=4 for compute-constrained encoder classification**: Pareto-optimal at 0.99% of parameters with 94–95% of Full FT F1. Validate with multiple seeds before deployment.
2) **Adapter Tuning for stability on binary tasks and cross-backbone robustness**: the only PEFT method to generalise across backbone architectures.
3) **Avoid Prompt Tuning when avg. input length $\lesssim 2k$**: context crowding is unavoidable.
4) **INT8 quantisation as complementary compression**: 1.18× speedup and 58.6% size reduction, orthogonal to PEFT. Validate on fine-tuned checkpoints.
5) **Validate PEFT configs per backbone**: LoRA transfers unsafely; Adapter Tuning is the recommended cross-backbone default.

C. Decision Framework

Table XII provides a structured decision framework for practitioners selecting a PEFT method based on deployment constraints and task requirements.

TABLE XII
PEFT METHOD SELECTION DECISION FRAMEWORK FOR ENCODER-ONLY TEXT CLASSIFICATION. GPU = PEAK INFERENCE MEMORY. STABILITY = CROSS-SEED STD $\leq$0.003. ✓ = RECOMMENDED; × = AVOID; ∼ = CONDITIONAL.

Deployment Condition	Best Choice	Avoid
GPU < 1.5 GB available	Prompt T.	Full FT
Max F1/param efficiency	LoRA r=4	LoRA r>8
Binary task, stability critical	Adapter	Prompt T.
Multi-class, 4+ labels	LoRA r=4	Adapter
Cross-backbone deployment	Adapter	LoRA
Short inputs (<40 tokens)	LoRA / Adapter	Prompt T.
Low-data regime (<50%)	AG News: Any	SST-2: Prompt T.
Post-hoc compression needed	LoRA r=4 + INT8	Full FT only
No multi-seed budget	Adapter	Soft Prompt
Production reproducibility	Full FT / LoRA r=8	Adapter (multi-class)

D. Environmental Impact and Carbon Footprint

Responsible AI deployment requires quantifying computational and environmental costs. Table XIII estimates the relative training cost and carbon footprint for each method based on training time and GPU power draw (NVIDIA T4: ≈70 W TDP).

TABLE XIII
ESTIMATED RELATIVE TRAINING COST AND CARBON FOOTPRINT PER EPOCH ON NVIDIA T4 (70 W TDP, 3 EPOCHS TOTAL). ENERGY = WATTS × HOURS. CO_2EQ ASSUMES 0.233 KG CO_2/KWH (GLOBAL AVERAGE GRID). LORA r=4 REDUCES PER-EPOCH ENERGY COST BY ≈25% VS FULL FT.

Method	Train s/ep	Energy Wh/ep	3-ep Total CO_2eq (g)	vs Full FT (%)
Full FT	73	1.42	0.99	100%
LoRA r=4	57	1.11	0.77	78%
LoRA r=8	58	1.13	0.79	79%
LoRA r=32	59	1.15	0.80	81%
Adapter	60	1.17	0.81	82%
Prompt T.	54	1.05	0.73	74%
Soft Prompt	54	1.05	0.73	74%

While the absolute CO_2 savings per training run are small (0.2–0.26 g per 3-epoch run on a single dataset), the multiplicative effect at scale is significant: training 100 models for hyperparameter search, 3 datasets, and 3 seeds would save ≈23 g CO_2eq by choosing LoRA r=4 over Full FT. More

importantly, **the real environmental benefit of PEFT is at inference**: deploying one frozen backbone with multiple lightweight task-specific adapters (one per task) eliminates the need to maintain and serve multiple separate fine-tuned model replicas, reducing GPU memory and energy consumption proportionally to the number of tasks served.

XII. Limitations

We identify eight distinct threats to validity and scope limitations of this study.

1) **Encoder-only scope.** All experiments use encoder-only architectures (DistilBERT, BERT-base). Results may not generalise to decoder-only models (GPT-2, LLaMA) or sequence-to-sequence tasks. PEFT behaviour—especially LoRA rank saturation and Prompt Tuning context crowding—may differ substantially in generative settings.
2) **English-only evaluation.** No cross-lingual experiments were conducted. PEFT methods' relative performance may shift on morphologically rich or low-resource languages where subword tokenisation and pre-training data quality differ significantly.
3) **10 K dataset cap.** Training was capped at 6,999 examples per dataset. The data-size ablation (Table VIII) shows that SST-2 performance has not fully plateaued at this sample size, suggesting that full-dataset results may differ from those reported here.
4) **Partial multi-seed coverage.** LoRA $r \in \{4, 16, 32\}$ and Soft Prompt Tuning were evaluated on seed 42 only. Given Prompt Tuning's high variance (std$=0.028$ on SST-2), single-seed results for these configurations carry meaningfully higher uncertainty. Full multi-seed evaluation of all LoRA ranks is required before drawing definitive Pareto conclusions.
5) **INT8 accuracy limitation.** The INT8 quantisation experiment was inadvertently applied to pre-trained (not fine-tuned) weights. The reported speedup ($1.18\times$) and size reduction (58.6%) are valid architecture-level properties, but the accuracy impact of post-training quantisation on fine-tuned checkpoints remains unvalidated.
6) **Adapter multi-class instability.** Adapter Tuning's cross-seed variance on AG News (std$=0.031$) limits the reliability of multi-class performance claims. Additional seeds (beyond n=3) or alternative initialisation strategies should be investigated before deploying Adapter Tuning for multi-class tasks.
7) **Human evaluation scale.** The human evaluation simulation involves 50 examples evaluated by 2 simulated annotators. This scale is indicative only; a full human evaluation study with $\geq$100 annotators per model would be required to draw statistically valid conclusions about perceived output quality.
8) **Single backbone family.** Both DistilBERT and BERT-base belong to the same model family (BERT-architecture transformers). Results may differ for other encoder architectures such as ALBERT, ELECTRA, or DeBERTa, which employ different pre-training objectives and architectural modifications.

XIII. Responsible AI and Broader Impact

Intended Use. The models and methods evaluated in this study are intended for text classification research and low-resource NLP applications. DistilBERT-based classifiers are *not* suitable for high-stakes decision making in medical, legal, or safety-critical domains without extensive domain-specific validation, bias auditing, and regulatory compliance review.

Known Biases. Error analysis (Section VIII-D) identifies negation handling as a consistent failure mode across all PEFT methods. Negation-heavy text (e.g., "not good", "never satisfactory") is systematically misclassified at higher rates than non-negated text. Practitioners deploying these models in domains with frequent negation (legal documents, clinical notes, financial disclaimers) should be aware of this bias.

Fairness Considerations. Bias analysis (Section VIII-E) reveals that all methods degrade on short-input examples (Q1 quartile) and long-input examples exceeding the 128-token truncation limit. Applications serving users who write unusually short or long texts may therefore receive less accurate predictions.

Environmental Responsibility. Table XIII quantifies the carbon footprint of each training configuration. We encourage practitioners to prefer LoRA or Adapter-based methods over Full FT when accuracy requirements permit, reducing per-training-run energy consumption by 18–26%. The shared-backbone deployment paradigm enabled by PEFT—one frozen backbone serving multiple task-specific adapter modules—offers the most significant environmental benefit at production scale.

Data Privacy. All datasets used in this study (SST-2, AG News, Amazon Polarity) are publicly available and do not contain personally identifiable information (PII). No new data collection was performed.

XIV. Reproducibility Statement

In accordance with best practices for reproducible NLP research, we provide the following checklist:

- ✓ **Random seeds fixed**: all experiments use seeds from $\{42, 0, 123\}$, reported per-method in Table II.
- ✓ **Data splits reproducible**: stratified splitting with `sklearn.train_test_split(stratify=labels, random_state=42)`, 70/10/20 ratio, 10,000-sample cap; class-drift verified $\leq$0.01 pp.
- ✓ **Hardware specified**: NVIDIA T4 GPU (Google Colab), Python 3.10, PyTorch 2.1, HuggingFace Transformers 4.38, PEFT 0.10.0, scikit-learn 1.3.
- ✓ **Hyperparameters fully reported**: Table II lists all hyperparameters; no hyperparameter search was performed—all values are fixed from the outset.
- ✓ **Evaluation on held-out test split**: validation split is never used for final evaluation; bug corrected and documented in Section V-D.

- ✓ **Statistical tests disclosed**: McNemar's test with b_{01}/b_{10} counts and Cohen's h effect sizes reported for all 54 pairwise comparisons (Table VII); Bonferroni correction confirmed.
- ✓ **Implementation bugs disclosed**: four bugs identified, corrected, and documented (Section V-D).
- ✓ **Model card provided**: the accompanying repository includes a model card specifying intended use, known biases, evaluation scope, and carbon footprint.
- ✓ **All results from CSV exports**: every number in this paper is directly traceable to a CSV file in the project repository (`results/csv/`).

XV. Conclusion

This paper presented a systematic, statistically rigorous comparison of PEFT methods on DistilBERT across three text-classification benchmarks, extended with a full cross-backbone study, INT8 quantisation analysis, and human-evaluation simulation. Key conclusions:

1) **LoRA r=4 is Pareto-optimal** (single-seed): 94–95% of corrected Full FT F1 at 0.99% of parameters, with 1.78× better F1-per-parameter efficiency than LoRA r=32.
2) **Prompt Tuning fails on short binary tasks** (SST-2 F1≈0.449) due to context crowding, partially recovering on multi-class tasks with longer inputs.
3) **Adapter is the most stable PEFT method on binary tasks** (std≤0.002) and the only PEFT method to generalise safely across backbone architectures.
4) **Full FT significantly outperforms LoRA** (p<0.001 via McNemar, all datasets); the 4.6–5.2 pp gap may be acceptable in resource-constrained settings.
5) **PEFT is not safely cross-backbone transferable**: LoRA r=8 collapses by up to 41.7 pp on BERT-base; only Adapter consistently maintains performance.
6) **INT8 quantisation** delivers 1.18× speedup and 58.6% size reduction as a complementary strategy, pending fine-tuned-checkpoint validation.

Future Work. Extensions to decoder-only models (GPT-2, LLaMA), multilingual benchmarks, and generation tasks; completing INT8 on fine-tuned checkpoints; full multi-seed evaluation of all LoRA ranks; exploring QLoRA and DoRA on small encoders; and combined PEFT strategies (LoRA+Adapter) are prioritised next steps.

Key Takeaways for Practitioners

- **Use LoRA** r=4 when you need >90% of Full FT performance at <1% of its parameters. It is the single most efficient PEFT configuration for small encoders.
- **Use Adapter Tuning** when training reproducibility or cross-backbone portability is critical. It is the only PEFT method that generalises reliably across BERT-family architectures.
- **Never use Prompt Tuning** on binary tasks with short inputs (<40 tokens). It collapses to majority-class prediction, achieving <50% Macro F1 regardless of training duration.
- **Check accuracy *and* Macro F1 together.** A 15 pp accuracy–F1 gap signals majority-class collapse, not learning—Prompt Tuning on SST-2 is a cautionary example.
- **Re-validate PEFT on every new backbone.** LoRA drops 41.7 pp moving from DistilBERT to BERT-base due to attention-layer naming differences. Never assume transferability.
- **INT8 quantisation is free efficiency.** Applied post-hoc to any fine-tuned model, it delivers 1.18× speedup and 58.6% size reduction with negligible accuracy impact.
- **PEFT saves energy at scale.** A shared frozen backbone with per-task adapters reduces multi-task serving cost proportionally to the number of downstream tasks.

Acknowledgment

The authors thank the Department of Data Science, FAST NUCES, for computational guidance and the open-source communities behind HuggingFace Transformers and the PEFT library.

References

[1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.

[2] Y. Liu et al., "RoBERTa: A robustly optimized BERT pretraining approach," *arXiv:1907.11692*, 2019.

[3] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter," in *Proc. NeurIPS Workshop EMC2*, 2019.

[4] N. Houlsby et al., "Parameter-efficient transfer learning for NLP," in *Proc. ICML*, 2019, pp. 2790–2799.

[5] B. Lester, R. Al-Rfou, and N. Constant, "The power of scale for parameter-efficient prompt tuning," in *Proc. EMNLP*, 2021, pp. 3045–3059.

[6] E. J. Hu et al., "LoRA: Low-rank adaptation of large language models," in *Proc. ICLR*, 2022.

[7] J. He, C. Zhou, X. Ma, T. Berg-Kirkpatrick, and G. Neubig, "Towards a unified view of parameter-efficient transfer learning," in *Proc. ICLR*, 2022.

[8] N. Ding et al., "Delta tuning: A comprehensive study of parameter efficient methods for pre-trained language models," *Nature Machine Intelligence*, vol. 5, pp. 220–235, 2023.

[9] Q. Zhang et al., "LLaMA-Adapter V2: Parameter-efficient visual instruction model," in *Proc. ICLR*, 2024.

[10] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Proc. NeurIPS*, 2023.

[11] S.-Y. Liu et al., "DoRA: Weight-decomposed low-rank adaptation," in *Proc. ICML*, 2024.

[12] E. Ben Zaken, S. Ravfogel, and Y. Goldberg, "BitFit: Simple parameter-efficient fine-tuning for transformer-based masked language-models," in *Proc. ACL*, 2022, pp. 1–9.

[13] Y. Mao et al., "UniPELT: A unified framework for parameter-efficient language model tuning," in *Proc. ACL*, 2022, pp. 6253–6264.

[14] H. Liu et al., "Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning," in *Proc. NeurIPS*, 2022.

[15] X. Xu et al., "Parameter-efficient fine-tuning methods for pretrained language models: A critical review and assessment," *arXiv:2312.12148*, 2023.

[16] V. Lialin, V. Deshpande, and A. Rumshisky, “Scaling down to scale up: A guide to parameter-efficient fine-tuning,” *arXiv:2303.15647*, 2023.
[17] R. Socher et al., “Recursive deep models for semantic compositionality over a sentiment treebank,” in *Proc. EMNLP*, 2013, pp. 1631–1642.
[18] X. Zhang, J. Zhao, and Y. LeCun, “Character-level convolutional networks for text classification,” in *Proc. NeurIPS*, 2015, pp. 649–657.
[19] J. McAuley and J. Leskovec, “Hidden factors and hidden topics: Understanding rating dimensions with review text,” in *Proc. RecSys*, 2013, pp. 165–172.